# MACHINE LEARNING LABORATORY

### Experiment No. 02

# Implementation of Simple Linear Regression

### and Evaluation of a Regression Model

## 1. Aim

To implement a Simple Linear Regression model for predicting a continuous target variable, generate predictions on test data, and evaluate the performance of the trained model using appropriate regression evaluation metrics.

## 2. Objectives

After completing this experiment, students will be able to:

1. Understand the concept of Simple Linear Regression.
2. Identify the feature and target variables for a regression problem.
3. Split the dataset into training and testing sets.
4. Train a Linear Regression model using training data.
5. Generate predictions for unseen test data.
6. Compare actual and predicted values.
7. Calculate regression evaluation metrics.
8. Interpret the performance of a regression model.
9. Visualize the fitted regression line and model predictions.

## 3. Prerequisites

Students should have basic knowledge of:

- Python programming
- NumPy
- Pandas
- Matplotlib
- Basic statistics
- Train-Test Split
- Basic Machine Learning workflow

# 4. Software Requirements

- Google Colab / Jupyter Notebook
- Python 3.x
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

# 5. Theory

## 5.1 Linear Regression

Linear Regression is a supervised machine learning algorithm used to predict a **continuous numerical value**.

In Simple Linear Regression, the relationship between one independent variable (feature) and one dependent variable (target) is represented using a straight line.

The mathematical equation is:

$$y = \beta_0 + \beta_1 x \tag{1}$$

where:

- $x$ = input feature
- $y$ = target variable
- $\beta_0$ = intercept
- $\beta_1$ = coefficient or slope

The model learns the values of $\beta_0$ and $\beta_1$ from the training data.

For example, if we want to predict a student's examination score based on the number of hours studied, the model can be represented as:

$$\text{Score} = \beta_0 + \beta_1(\text{Hours Studied})$$

The objective is to find a line that provides predictions as close as possible to the actual observations.

## 5.2 Dataset Used in the Experiment

For this experiment, a small dataset containing the number of hours studied by students and their corresponding examination scores is used.

| Hours Studied | Exam Score |
|---|---|
| 1 | 35 |
| 2 | 40 |
| 3 | 50 |
| 4 | 55 |
| 5 | 65 |
| 6 | 70 |
| 7 | 75 |
| 8 | 85 |
| 9 | 90 |
| 10 | 95 |

Table 1: Hours Studied and Exam Score

In this dataset:

$$X = \text{Hours Studied}$$

$$y = \text{Exam Score}$$

Since there is only one input feature, this is called **Simple Linear Regression**.

## 5.3 Regression Evaluation Metrics

After training the model, its predictions must be evaluated to determine how well the model performs.

Let:

$$y_i = \text{actual value}$$

and

$$\hat{y}_i = \text{predicted value}$$

### Mean Absolute Error (MAE)

MAE represents the average absolute difference between actual and predicted values.

$$MAE = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

A lower MAE indicates that the predictions are, on average, closer to the actual values.

### Mean Squared Error (MSE)

MSE calculates the average of the squared prediction errors.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Because the errors are squared, larger errors have a greater influence on MSE.

### Root Mean Squared Error (RMSE)

RMSE is the square root of MSE.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

RMSE is expressed in the same units as the target variable.

### $R^2$ Score

The coefficient of determination, $R^2$, indicates how much of the variation in the target variable is explained by the regression model.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the actual target values.
A value of $R^2$ closer to 1 generally indicates a better fit to the data.

# 6. Procedure

1. Import the required Python libraries.
2. Create the dataset containing Hours Studied and Exam Score.
3. Explore the dataset using Pandas.
4. Visualize the relationship between Hours Studied and Exam Score.
5. Separate the feature variable $X$ and target variable $y$.
6. Split the dataset into training and testing sets.
7. Create a Linear Regression model.
8. Train the model using the training data.
9. Obtain predictions for the test data.
10. Compare actual and predicted values.
11. Calculate MAE, MSE, RMSE, and $R^2$.
12. Visualize the fitted regression line.
13. Interpret the obtained evaluation metrics.

# 7. Implementation

## 7.1 Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score
```

## 7.2 Create the Dataset

```
data = {
    'Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Score': [35, 40, 50, 55, 65, 70, 75, 85, 90, 95]
}

df = pd.DataFrame(data)

df
```

## 7.3 Explore the Dataset

Display the structure and basic statistical information of the dataset.

```
print(df.info())

print(df.describe())
```

Students should identify:

- Number of observations
- Feature column
- Target column
- Data types
- Minimum and maximum values

## 7.4 Visualize the Relationship

A scatter plot can be used to observe the relationship between the feature and target variable.

```
plt.scatter(df['Hours'], df['Score'])

plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.title('Hours Studied vs Exam Score')

plt.show()
```

The scatter plot provides an initial understanding of whether a linear relationship may exist between the feature and target.

## 7.5 Separate Feature and Target Variables

The feature and target variables are separated before training the model.

```
X = df[['Hours']]
y = df['Score']

print("Feature:")
print(X)

print("Target:")
print(y)
```

Here, $X$ contains the input feature and $y$ contains the target values.

## 7.6 Split the Dataset

The dataset is divided into training and testing sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))
```

The training data is used to learn the relationship between the feature and target, while the test data is used to evaluate the model on unseen observations.

## 7.7 Create and Train the Linear Regression Model

```
model = LinearRegression()

model.fit(X_train.values.reshape(-1,1), y_train)
```

The `fit()` method trains the model using the training data.
The trained model learns the coefficient and intercept of the regression line.

## 7.8 Obtain the Model Parameters

```
print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)
```

The learned regression equation can be written as:

$$\hat{y} = \beta_0 + \beta_1 x$$

where the values of $\beta_0$ and $\beta_1$ are obtained from the trained model.

## 7.9 Generate Predictions

Use the trained model to predict the target values for the test data.

```
y_pred = model.predict(X_test)

print("Predicted values:")
print(y_pred)
```

The predictions generated by the model are represented by $\hat{y}$.

## 7.10 Compare Actual and Predicted Values

```
comparison = pd.DataFrame({
    'Actual': y_test.values,
    'Predicted': y_pred
})

print(comparison)
```

The difference between an actual value and its corresponding predicted value represents the prediction error:

$$Error = y_i - \hat{y}_i$$

## 7.11 Calculate Evaluation Metrics

```
mae = mean_absolute_error(y_test, y_pred)

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

r2 = r2_score(y_test, y_pred)

print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

### 7.12 Visualize the Regression Line

```
plt.scatter(X, y, label='Actual Data')

plt.plot(
    X,
    model.predict(X),
    label='Regression Line'
)

plt.xlabel('Hours Studied')
plt.ylabel('Exam Score')
plt.title('Simple Linear Regression')

plt.legend()
plt.show()
```

The scatter points represent the observed data, while the straight line represents the relationship learned by the Linear Regression model.

## 8. Results

The Simple Linear Regression model was successfully trained using the training dataset. Predictions were generated for the test dataset and the model was evaluated using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and $R^2$ score.

Students should record the obtained values in the following table.

| Evaluation Metric | Obtained Value |
|---|---|
| MAE | |
| MSE | |
| RMSE | |
| $R^2$ Score | |

Table 2: Regression Model Evaluation

## 9. Interpretation

Answer the following questions based on the obtained results:

1. What does the coefficient of the Linear Regression model represent?
2. What does the intercept represent?
3. What does the MAE value tell us about the model?
4. Why is MSE more sensitive to large errors?
5. Why is RMSE expressed in the same units as the target variable?

6. What does the obtained $R^2$ score indicate?
7. Which observations were predicted accurately?
8. Which observations have larger prediction errors?

# 10. Conclusion

Simple Linear Regression was implemented to predict examination scores from the number of hours studied. The model was trained using training data and evaluated using unseen test data. Different regression evaluation metrics were used to quantify the prediction error and assess the performance of the model.

# 11. Viva Questions

1. What is supervised learning?
2. What is the difference between regression and classification?
3. What is Simple Linear Regression?
4. Why is the algorithm called "linear" regression?
5. What is the role of the coefficient $\beta_1$?
6. What is the role of the intercept $\beta_0$?
7. What is the purpose of train-test split?
8. What does the `fit()` method do?
9. What does the `predict()` method do?
10. What is the difference between actual and predicted values?
11. What is MAE?
12. What is MSE?
13. What is RMSE?
14. What does $R^2$ measure?
15. Which metric is more affected by large errors: MAE or MSE?
16. Can a Linear Regression model be used for classification?
17. What happens if the relationship between $X$ and $y$ is not approximately linear?

# 12. Additional Exercises

1. Change the values in the dataset and observe how the regression line changes.
2. Change the `test_size` from 0.2 to 0.3 and compare the evaluation metrics.
3. Change the `random_state` and observe whether the model performance changes.
4. Add a new observation to the dataset and retrain the model.
5. Predict the examination score for a student who studied for 7.5 hours.
6. Calculate the prediction error for each test observation.
7. Compare MAE, MSE, and RMSE and explain which metric is easier to interpret in terms of examination score.